

Yêu cầu:

Tìm 3 API public thật viết về một domain quan tâm

Gợi ý: GitHub REST, OpenWeather, ExchangeRate-API, Spotify Web API, public-apis repo trên GitHub. Mỗi API trả lời: loại nào REST/GraphQL/gRPC? base URL? auth method? có versioning? mỗi resource identifier là số hay UUID?

Đọc Higginbotham Ch.1 Geewax Ch.1

Tập trung vào: định nghĩa API, resource, và 5 đặc điểm của một API tốt. So sánh với 5 đặc điểm đã học trong slide 05 — có điểm nào Geewax/Higginbotham bổ sung?

Hoàn thiện bài 6 + mở rộng

Sau khi CRUD chạy được, thêm: (a) tìm kiếm GET /books?q=...; (b) sort theo ?sort=title; (c) bắt buộc field year là số ≥ 1900 .

Bài làm:

Google Maps API

- + Kiến trúc: REST
- + <https://maps.googleapis.com/maps/api/>
- + API Key: truyền qua URL Query (?key=YOUR_KEY) trên API truyền thống
- + Có versioning, định danh trên đường dẫn v1/v2
- + Resource Identifier: chuỗi riêng của gg

Gemini API

- + Kiến trúc: REST
- + <https://generativelanguage.googleapis.com/v1beta/>
- + API key
- + Có versioning
- + Resource Identifier: model: chuỗi định dạng models/gemini-1.5-flash

Github API

- + Cả rest api và graphQL api
- + <https://api.github.com/> cho rest và <https://api.github.com/> graph
- + Personal Access Token (PAT) / OAuth 2.0 truyền qua http header. App authentication
- + Có versioning
- + Resource Identifier: rest sử dụng số tự nhiên tăng cho các đối tượng kết hợp slug/name cho tài nguyên; chuỗi mã hóa base64 riêng của graph

Higginbotham Ch.1 Geewax Ch.1

API

Geewax: Ranh giới tương tác giữa client và server qua mạng, cho phép thao tác dữ liệu mà không cần access trực tiếp vào mã nguồn hay data bên trong

Higginbotham: digital contract và capability giúp các hệ thống chia sẻ khả năng nghiệp vụ riêng biệt

Resource:

Đóng vai trò là danh từ thông qua các phương thức giao tiếp chuẩn (GET, DELETE, PUT, POST) hoặc tùy chỉnh

5 đặc điểm của một API tốt

- + Operational (tính tin cậy)
- + Expressive (tự giải thích)
- + Simple (đơn giản)
- + Predictable (tính nhất quán)
- + Maintainable (dễ bảo trì)

⇒ Tài liệu bổ sung so với slides:

- + Nhấn mạnh hợp đồng phải có tính nhất quán cao, đặt tên phải đồng bộ để dev có thể đọc 1 đoán và sử dụng 10
- + Api cần được thiết kế đơn giản nhất để dev truyền vào ko quá nhiều tham số k cần thiết
- + Tư duy api là sản phẩm phục vụ business

```
@app.route("/books", methods=["GET"])
def list_books():
    q = request.args.get("q", "").strip().lower()
    sort_by = request.args.get("sort", "").strip()
    # min > 1900
    try:
        min_year = int(request.args.get("min_year", 1900))
    except ValueError:
        return jsonify({"error": "min_year phải là số nguyên"}), 400
    if min_year < 1900:
        return jsonify({"error": "min_year phải >= 1900"}), 400
    result = [
        b for b in BOOKS
        if (q in b["title"].lower() or q in b["author"].lower())
        and b.get("year", 1900) >= min_year
    ]
    # sort
    if sort_by and result and sort_by in result[0]:
        result = sorted(result, key=lambda x: x[sort_by])
    return jsonify(result), 200
```